01f — How Browsers Work

Key Concepts

Term	Definition
DOM	Document Object Model — tree of HTML elements built by the browser
CSSOM	CSS Object Model — tree of styles built from all CSS sources
Render Tree	DOM + CSSOM combined — only visible elements with computed styles
Layout (Reflow)	Calculates exact position and size of every element
Paint	Fills in pixels — colors, borders, text, images
Compositing	GPU merges layers (fixed headers, animations) into final frame
Critical Rendering Path	The sequence: HTML → DOM → Render Tree → Layout → Paint
Reflow	Layout recalculation triggered by DOM/size changes. Expensive.
Repaint	Pixel redraw without layout change. Cheaper.
V8	Chrome/Node.js JavaScript engine

Browser Components

Component	Role
User Interface	Address bar, navigation buttons
Browser Engine	Coordinates UI and rendering engine
Rendering Engine	Parses HTML/CSS, builds page
Networking Layer	HTTP requests, DNS, caching
JavaScript Engine	Executes JS (V8 in Chrome)
Data Storage	Cookies, localStorage, IndexedDB

How It Works

Full flow after DNS resolves:

1. TCP connection opened to server IP
2. TLS handshake (HTTPS) — establishes encrypted connection
3. Browser sends `HTTP GET /index.html`
4. Server returns HTML (200 OK)
5. Rendering engine parses HTML → **DOM Tree**
6. CSS downloaded and parsed → **CSSOM Tree**

7. DOM + CSSOM → **Render Tree** (visible elements + computed styles only)
8. **Layout** — calculates position and size of every element
9. **Paint** — fills pixels: colors, text, borders, images
10. **Compositing** — GPU merges layers into final frame
11. JS engine downloads and executes scripts → may trigger reflow/repaint

---

Critical Rendering Path

```
HTML → DOM ──┐
             ├── Render Tree → Layout → Paint
CSS → CSSOM ─┘
```

Render-blocking resources:

- `<script>` without `async`/`defer` — stops HTML parsing until downloaded + executed
- CSS — blocks rendering until fully parsed

Script Loading Strategies

Strategy	Behaviour	Use Case
Normal `<script>`	Blocks HTML parsing	Avoid for large scripts
Bottom of `<body>`	HTML parsed first, then blocks	Old approach
`defer`	Downloads in parallel, executes after DOM ready	App scripts
`async`	Downloads in parallel, executes immediately	Independent scripts (analytics)

```
Normal:  HTML ──stops── download ── execute ── resumes
Defer:   HTML ──────────────────────────────── execute after DOM
Async:   HTML ──────────── execute immediately (whenever ready)
```

Reflow vs Repaint

	Reflow	Repaint
Triggered by	DOM changes, resize, font change	Color, visibility, background change
Cost	Expensive — cascades to surrounding elements	Cheaper
Example	Changing width of a div	Changing background color

Browser Storage

Type	Size	Persists	Sent with HTTP?
Cookie	~4KB	Yes (expiry set)	Yes — every request
localStorage	~5MB	Yes	No
sessionStorage	~5MB	Until tab closes	No
IndexedDB	Large	Yes	No

Industry Examples

Company	Usage
Google V8	JS engine in Chrome and Node.js — same engine for browser and server
Google Core Web Vitals	LCP, FID, CLS — rendering performance metrics used in search ranking
Facebook	Code-splits JS bundles so only initial-view code loads, reducing time-to-paint

Interview Questions

Q: What is the Critical Rendering Path? A: The sequence a browser must complete before pixels appear — HTML → DOM, CSS → CSSOM, combined into Render Tree → Layout → Paint.

Q: What is the difference between reflow and repaint? A: Reflow recalculates layout (positions/sizes) and is expensive because it cascades. Repaint redraws pixels without layout change and is cheaper.

Q: What is the difference between `async` and `defer`? A: Both download in parallel with HTML parsing. `defer` executes after DOM is ready, in order. `async` executes immediately when downloaded, regardless of DOM state.

Q: Why is JavaScript render-blocking by default? A: JS can modify the DOM, so the browser stops parsing HTML until the script runs, to avoid building a DOM that will immediately change.

Q: What's the difference between localStorage and a cookie? A: localStorage stays in the browser and is never sent to the server. Cookies are sent with every HTTP request, making them useful for auth sessions but expensive for large data.